GENOX: Exploiting Thread Signals for Dynamic CPU Scaling in Genomics Workflows

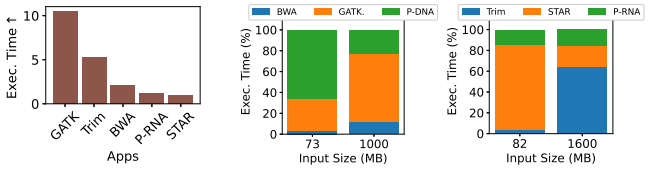
***Abstract*—DAG-structured genomics workflows process gigabytes to terabytes of sequencing data in shared clusters, where efficient CPU allocation is a critical decision in resource management. Suboptimal allocations can significantly delay workflow executions and increase computational costs. However, existing CPU allocation approaches are often insufficient for genomic workflows due to their unique challenges: input-dependent performance variability, contention-induced interference, and dynamic workflow branching.**

In this work, we first characterize production-level workflows for processing DNA and RNA sequencing data, identifying three critical characteristics: inflatable parallelism (thread surges up to 96×), stall heterogeneity (up to 2× slowdowns) and millisecond-scale idle bursts that make utilization often unreliable. These findings confirm that effective CPU management should satisfy three requirements: 1) responsiveness to thread bursts, 2) robustness under contention, and 3) adaptiveness to workflow dynamics. Motivated by our characterization study, we developed GENOX, an automated CPU quota management system that leverages thread counts as its primary signal for CPU scaling, rather than traditional metrics like utilization. Operating at 5ms intervals through standard Linux cgroups, GENOX responds immediately to changes in CPU demand phases while remaining stable under contention. Evaluation results show that GENOX outperforms state-of-the-art baselines, as it reduces application execution time by up to 50%, eliminates 200% straggler slowdowns, and improves workflow completion by 21%. These evaluation results confirm that thread-based scaling effectively addresses CPU allocation challenges in genomics workflows.

I. INTRODUCTION

Genomics workflows are composed of a set of bioinformatics applications through DAG-structured pipelines, processing gigabytes to terabytes of omics data while competing for CPU resources in shared clusters [1]–[3]. As large-scale workflows often consume thousands of CPU hours [4], [5], CPU allocation becomes more than merely an optimization problem. Such an allocation decision can have significant cost implications on the large-scale processing of genomic data. For example, suboptimal CPU allocations propagate through the pipeline, increasing computational costs.

However, automatically determining the right CPU quota remains a significant challenge due to the unique characteristics of genomics workflows. For example, genomics workflows often exhibit substantial performance variability that can be hard to predict, depending on input characteristics. As shown in Fig. 1a, increasing input size causes execution times for some genomics applications (*e.g.*, STAR [6]) to scale linearly, whereas others (*e.g.*, GATK-br [7]) can experience up to a 10× increase in runtime. This increase in workflow execution time often results from dynamic changes in the workflow's critical path, which is the sequence of operations that dominates total runtime, depending on the input. As shown in Fig. 1b and 1c, an application that takes $\leq$ 10% of runtime on small inputs may consume up to 60% on larger inputs. Unfortunately, these shifts are difficult to predict without executing each specific dataset.



(a) Relative Exec. Time ↑ (b) DNA-Seq. (c) RNA-Seq.

Fig. 1. **Input-dependent performance variability.** *Genomics applications exhibit varying runtime growth with input size, up to 10× differences (1a) and shifting critical paths in DNA-Seq (1b) and RNA-Seq (1c).*

Furthermore, since multiple workflows are executed concurrently in production environments [8], [9], contention-induced slowdowns become increasingly difficult to detect and mitigate. Under contention, traditional CPU utilization metrics often fail to accurately capture true CPU demand. These metrics may report low utilization during I/O stalls or millisecond-scale idle bursts. I/O stalls occur when threads are blocked, waiting for disk or network operations to complete. Millisecond-scale idle bursts occur when applications rapidly alternate between computation and I/O phases at fine time scales, resulting in rapid fluctuations in CPU usage. Therefore, these patterns can make utilization metrics misleading, as they may indicate low CPU usage even when applications have pending computational work. Finally, genomics workflows can include conditional branches, where certain applications run only if the input data meets specific conditions [8], [10]. For example, low-quality genomics data triggers additional refinement steps using data quality control tools like FastQC [11] or Trimmomatic [12], whereas high-quality data bypasses these stages entirely. As a result, resource demands can shift suddenly and unpredictably during execution.

Current CPU allocation approaches, *e.g.*, utilization-based CPU allocators [13]–[15], ML-based predictors [16]–[18], or kernel-integrated approaches [19], [20], are often inadequate for genomics workloads. These approaches either react with a delay to dynamic CPU demands [21], [22] or require extensive historical data for predictive model training [23], [24] that *may be infeasible* to obtain (given the high variability of genomic input data) or demand application/kernel-level modifications, which are incompatible with existing bioinformatics tools.

To understand these challenges quantitatively, we conducted an in-depth characterization study of widely used computational genomics workflows, including DNA sequencing (often abbreviated to DNA-Seq) and RNA sequencing (RNA-Seq) [25]. Our measurements reveal three key characteristics. Specifically, we identify *inflatable parallelism*, where thread counts surge up to 96×; measure *stall heterogeneity*, causing

up to 2.1× slowdown from resource contention; and observe *millisecond-scale idle bursts*, which make utilization metrics inherently unreliable for quota decisions under contention.

These findings guide us to establish three key requirements for managing CPU quotas in genomics workflows. A solution must be (i) **responsive to thread bursts**, responding immediately to inflatable parallelism without waiting for (or confirming) utilization changes; (ii) **robust under contention**, maintaining accurate quota decisions despite heterogeneous stalls and idle bursts; and (iii) **adaptive to workflow dynamics**, adjusting to conditional branches and varying application sets without manual intervention.

We present GENOX (GENOmics eXecution), an automated CPU scaling system that addresses these requirements by exploiting thread signals as the primary indicator of compute demand. Our key insight is that thread count can serve as a primary indicator of resource needs in genomics applications, providing immediate, stall-agnostic signals that remain stable even under contention. GENOX is composed of three components: (1) a **thread-based scaler**, which allocates CPU quota proportionally to active threads, detecting CPU demand phase changes immediately as threads are created; (2) a **fine-grained thread monitor**, which operates at 5 ms intervals to capture millisecond-scale dynamics while remaining robust against misleading utilization signals; and (3) a **container-native controller**, which seamlessly incorporates with genomics workflows/applications via standard cgroup interfaces.

We evaluate GENOX in production-like environments using widely adopted genomics workloads, including both microbenchmarks with representative applications and macro-scale experiments under realistic contention scenarios. Our micro-benchmarks with three representative applications (*e.g.*, BWA [26], STAR, BamSorMaDup [27]) confirm that GENOX effectively addresses all three core requirements (responsiveness, robustness, adaptiveness). GENOX reacts immediately to thread bursts in BWA's inflatable parallelism phases, maintains stable allocations despite STAR's millisecond-scale idle bursts, and efficiently adapts to BamSorMaDup's constant parallel-thread execution, collectively validating our design principles.

Moreover, macro-level evaluation shows that GENOX consistently outperforms state-of-the-art baselines [13]–[15] under realistic contention, achieving up to 50% reduction in execution time and minimizing straggler slowdowns that reach 200% in baseline approaches. At the workflow level, GENOX improves end-to-end workflow completion time by over 20%.

This work has the following research contributions.

1) **Comprehensive characterization of genomics workflow dynamics**, identifying inflatable parallelism, stall heterogeneity, and millisecond-scale idle bursts as fundamental challenges for CPU allocation (§III).
2) **Design of a measurement-driven CPU quota allocation system** that uses threads as a stable, leading signal of compute demand, with fine-grained monitoring at 5ms intervals to detect true demand while minimizing resource-demand noise (§IV-B, §IV-C).

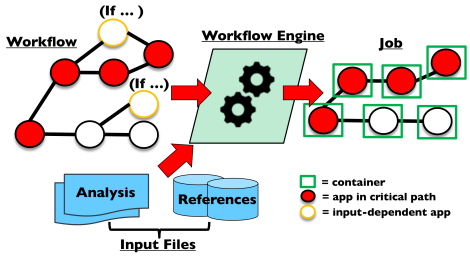


Fig. 2. **Example of genomics workflow execution.** *A workflow DAG with input-dependent steps executes using given analysis and reference files. Some steps are skipped based on input, and each active app runs within a container.*

3) **Non-invasive, production-ready implementation** that integrates containerized workflow engines using only standard cgroup interfaces (§IV-D).
4) **Comprehensive and thorough evaluation confirming GENOX's performance improvements** over utilization-based approaches across diverse genomics workloads under realistic contention scenarios (§V).

II. BACKGROUND AND MOTIVATION

Modern genomics workflows integrate dozens of containerized bioinformatics tools into pipelines that process terabytes of biological data. Workflow engines (*e.g.*, Nextflow [28], CWL [29]) coordinate task execution, while containers provide resource isolation [1], [8], [30]. We outline how such workflows operate in production environments and why CPU quota allocation is a key performance concern. Fig. 2 illustrates how these components interact within production environments.

A. Genomics Workflow Background

Workflow. A genomics workflow consists of interdependent steps (*e.g.*, alignment [31], variant calling [32], annotation [33]) represented as a DAG [10], where nodes are containerized applications, and edges are file-based dependencies. Workflows are described using languages, *e.g.*, CWL, WDL [34], or Nextflow DSL. These languages support optional inputs and conditional branches, allowing the same workflow definition to activate different steps depending on the dataset.

Job. A job is a single workflow execution with a specific dataset. Optional inputs and conditional branches enable identical workflow definitions to produce different execution paths.

Deployment. Production workflows are executed by workflow engines (*e.g.*, Nextflow, Cromwell [35], CWLtool [36]) with container runtimes (Docker, Singularity [30], [37]). Workflow engines schedule tasks and manage dependencies, while containers enforce CPU and memory quotas via Linux cgroups, which we leverage for dynamic and runtime CPU allocation.

Input. Genomics workflows process multiple types of input data. One common type is reference files (*e.g.*, the human genome reference [38]), serving as a baseline for analysis. Another common type is data files that contain DNA or RNA sequencing data. These sequencing files are typically accompanied by metadata, such as quality scores and processing parameters. The characteristics of input data have a direct impact on a genomics workflow's execution time. For example, the input

data size determines the required computational resources (*e.g.*, the number of CPU cores and memory allocation size), and the input format (*e.g.*, FASTQ or BAM) affects the I/O patterns observed during workflow execution. In addition, the presence of optional inputs enables conditional execution paths within the workflow. Specifically, input-dependent branching, due to the optional inputs, can make identical workflows to execute different subsets of applications depending on the input datasets [8], [10]. This behavior introduces performance variability that complicates performance prediction. We further discuss this challenge in §III-A.

Critical path. A workflow's completion time depends on its critical path, which is the longest chain of dependent steps in the DAG [39]. While identifying critical paths is straightforward in controlled settings, production environments with input-dependent dynamic branching make this substantially more complex. The same workflow definition may activate different steps for different inputs, resulting in different critical paths. This variability complicates performance prediction and makes CPU quota allocation challenging in practice.

Alignment workflow. Alignment matches DNA or RNA sequencing reads to a reference genome [40]. While the core algorithm is string matching, production workflows add extensive preprocessing, quality control, and metrics collection that significantly impact runtime. These alignment jobs are among the most resource-intensive tasks in genomics, often consuming tens of CPU cores and hundreds of gigabytes of memory for days or weeks. Alignment jobs typically run in groups of 50 – 100, quickly saturating cluster resources with thousands of cores and huge memory demand. This makes optimizing alignment jobs critical for cost-efficient large-scale genomics processing and drives the need for more effective CPU quota allocation strategies.

B. CPU Quota Allocation in Production Practice

Genomics workflows execute on shared clusters where multiple workloads compete for CPU resources. Current allocation schemes struggle with both over-allocation (wasting resources) and under-allocation (slowing down executions due to insufficient resources) problems, compounded by genomics workflows' runtime variability. Existing systems employ either static allocation, where resources are fixed at submission time (*e.g.*, SLURM [41], PBS [42], [43]), or dynamic allocation, where CPU quotas are adjusted based on utilization (*e.g.*, Kubernetes HPA [44], YARN [45]). However, both fail to handle genomics-specific challenges. For example, static allocation cannot adapt to runtime variations, and dynamic approaches suffer from inherent lag during demand bursts. We thus conduct a measurement-driven study in §III to characterize the variability, contention, and dynamic branching of genomics workflows to guide more effective CPU quota allocation.

III. Challenges and Measurement

The unique runtime characteristics of genomics workflows present significant challenges for effective CPU quota allocation. To investigate these challenges, we conducted measurements on two representative genomics workflows: DNA-Seq and RNA-Seq. These workflows integrate widely used genomics applications (*e.g.*, BWA, STAR, GATK, Trimmomatic) that dominate the runtime of production workloads.

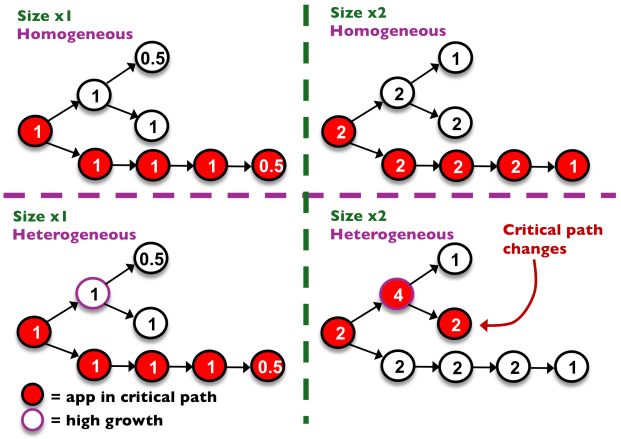


Fig. 3. **Critical path shifts with input size increase.** *'Top': Homogeneous scaling maintains the same critical path (as input size doubles). 'Bottom': Heterogeneous scaling causes the critical path to shift between applications (from blue to purple), complicating CPU quota allocation.*

We executed the workflows on the Chameleon Cloud [46], using servers with 96 CPU cores and 192 GB memory. Nextflow orchestrated execution with each application running in Docker containers. We monitored runtime metrics through kernel interfaces, collecting per-container CPU quotas, thread counts, and pressure stall information (PSI) [47]. We varied dataset sizes up to 12× to capture input-dependent effects.

Based on these measurements, we reveal three key challenges: (i) **input-dependent performance variability** (§III-A), (ii) **contention-induced slowdown** (§III-B), and (iii) **dynamic workflow branching** (§III-C). For each challenge, we conduct measurements to quantify its impact, reveal unique execution patterns, *e.g.*, *inflatable parallelism* and *stall heterogeneity*, and derive design insights that motivate our solution.

A. Input-dependent Performance Variability (C1)

Resource allocation for workflows should prioritize applications on the critical path to minimize their execution time. However, accurately predicting the completion time of genomics workflows remains challenging due to their complex execution characteristics. In particular, as input size increases, some workflows exhibit linear scaling, while others show up to a 10× increase in runtime beyond what linear scaling would predict. This heterogeneous, non-linear scaling shifts the critical path across applications, making it difficult to identify which applications will dominate runtime and complicating decisions for CPU quota allocation.

Fig. 3 explains this phenomenon conceptually. The top cases depict *homogeneous growth*, where all applications scale uniformly and the critical path remains unchanged as input size doubles. By contrast, the bottom cases depict *heterogeneous growth*, where differential scaling causes the critical path to

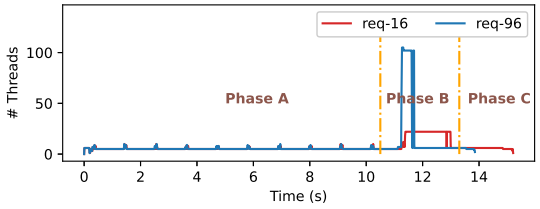


Fig. 4. **Inflatable parallelism ('Phase B') in BWA execution.** *Thread count dynamics across different phases for two configurations (16 and 96 threads). 'Phase A:' I/O-driven spikes, 'Phase B:' thread counts surge proportionally to user requests, 'Phase C:' a stable thread levels until completion. Phase B reveals configuration's impact on resource consumption and execution time.*

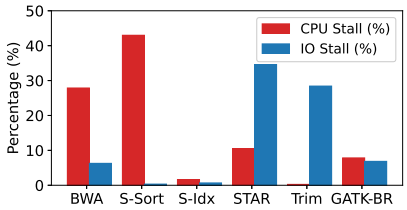


Fig. 5. **Stall heterogeneity across genomics applications.** *Distribution of CPU and I/O stall percentages showing diverse resource bottlenecks. Some applications are predominantly CPU-bound while others are I/O-bound, complicating utilization-based allocation under contention.*

shift to a different application. This heterogeneous behavior is what we have observed in production genomics workflows.

To quantify this challenge, we analyzed the 5 longest-running applications in both DNA-Seq and RNA-Seq workflows, measuring execution time growth under identical input size increases. Our results (Fig. 1 in §I) reveal that application run times increased by 22% to 232% relative to their baseline, with some experiencing up to 10× greater increase than others (Fig. 1a). This uneven scaling shifts the critical path (the application dominating runtime) as input size grows (Fig. 1b, 1c). In production environments where inputs reach hundreds of GB, these shifts become pronounced and further complicate CPU quota allocation.

Finding #1: Inflatable parallelism. Our measurements reveal that parallel genomics applications (*e.g.*, BWA) exhibit distinct execution phases corresponding to different computational stages. We consistently observe a phase with a phase with sharp thread count surges, where threads increase dramatically based on user configuration. We introduce the term *inflatable parallelism* to describe this behavior, where thread counts can suddenly expand based on user configuration, creating unpredictable resource demands. Fig. 4 illustrates this behavior for BWA, which proceeds through three phases: *Phase A*: recurrent small spikes, coinciding with I/O activity; *Phase B*: inflatable parallelism stage with thread surge proportional to the user configuration; *Phase C*: a drop followed by a stable number of threads until completion.

Only *Phase B* is sensitive to the user-specified configuration, causing variations in thread counts and overall execution time. This inflatable parallelism poses a significant challenge. *While proportional scaling may seem predictable, its impact on workflow runtime is substantial.* The inflated phase often dominates execution time and can shift the critical path as inputs grow. Existing utilization-based policies fail to timely detect these sudden inflations because utilization lags behind thread creation. By the time high utilization is detected, the application has already entered its resource-intensive phase, resulting in allocation delays and performance penalties.

B. Contention-induced Slowdown (C2)

Genomics workflows are often executed in batches [4], leading to the colocation of multiple applications on the same machine. Such colocated executions inevitably create contention, causing slowdowns as applications interfere with one another's resource usage. While resource allocators should

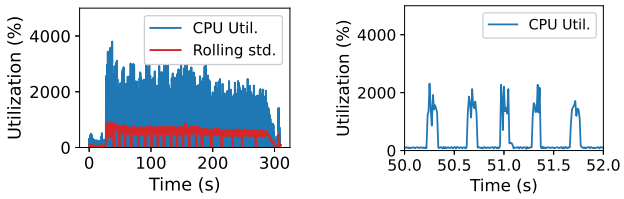
Table 1. **Contention-induced slowdowns across application pairs.** *Entries show normalized execution time; larger values indicate higher slowdown. Trimm.: Trimmomatic, P-DNA: Picard-DNA, P-RNA: Picard-RNA*

	Trimm.	STAR	P-RNA	BWA	GATK-br	P-DNA
Trimm.	1.4×	–	–	–	–	–
STAR	1.1×	2×	–	–	–	–
P-RNA	1.03×	1.01×	1.08×	–	–	–
BWA	1.07×	1.1×	1.05×	2.1×	–	–
GATK-br	1.08×	1.05×	1.05×	1.13×	2.02×	–
P-DNA	1.15×	1.04×	1.02×	1.11×	1×	1×

mitigate this contention, genomics applications exhibit two characteristics that make traditional utilization-based detection ineffective: (i) *stall heterogeneity* and (ii) *millisecond-scale repetitive idle bursts*.

Stall heterogeneity. Genomics applications differ widely in their dominant resource usage. Some applications are CPU-bound, others I/O-bound, and many alternate between the two. Fig. 5 shows the distribution of CPU and I/O stall time as reported by Linux PSI. Depending on the application, between 10% and 50% of execution time can be lost to stalls on different resources. This heterogeneity makes utilization an unreliable signal. For example, a high utilization reading may indicate genuine compute demand that would benefit from more CPUs, or it may simply be inflated by I/O-induced stalls and scheduling delays, in which case allocating additional CPUs would not improve performance. When such applications are colocated, interference leads to non-uniform slowdowns. Table 1 shows that co-execution slows down genomics workflows by up to 2.1×.

Millisecond-scale repetitive idle bursts. Genomics applications frequently interleave compute-intensive phases with short I/O operations, producing millisecond-scale repetitive idle bursts. Fig. 6a shows GATK's CPU utilization fluctuating sharply with 500% – 800% standard deviation. These bursts are not genuine demand shifts but artifacts of how applications



(a) CPU Utilization (b) Zoomed 2-sec. Window

Fig. 6. **Millisecond-scale idle burst dynamics in GATK.** *(a): CPU utilization shows large volatility over time. (b): fine-grained oscillations from alternating compute and I/O phases (b).*

alternate between I/O and computation (Fig. 6b). Under contention, such bursts further destabilize utilization as a scaling signal, creating the illusion of rapid demand changes.

Finding #2: Utilization misleads under contention. Taken two characteristics (stall heterogeneity and millisecond-scale repetitive idle bursts) together, these observations suggest that utilization is an unreliable indicator when applications are colocated. It cannot distinguish compute demand from I/O stalls, nor differentiate genuine idle periods from brief I/O waits. This finding necessitates a stall-agnostic, phase-aware signal for robust quota allocation.

C. Dynamic Workflow Branching (C3)

Finally, resource allocation for genomics workflows faces an additional challenge: dynamic workflow branching. Workflows often contain conditional branches that activate or skip certain applications based on specific input characteristics. These branches arise from conditional logic triggered by input type, input characteristics, input existence, or input metadata. Developers incorporate such logic to selectively perform pre-/post-processing or to tune algorithms based on input characteristics.

Our analysis of widely used harmonization workflows [25] shows that such branching is common, and its runtime impact varies with workflow complexity. In DNA Seq workflows, we observed that a conditional Picard step can add about 5% to execution time. While modest in isolation, the effect compounds in production workflows with multiple conditional steps, making accurate resource prediction increasingly difficult.

Finding #3: Dynamic thread demand from conditional execution. The set of active applications can change at runtime based on input characteristics, causing workflows to exhibit shifting resource demands and thread dynamics that cannot be predicted a priori. This unpredictability undermines static allocation strategies and necessitates runtime adaptivity.

D. Measurement Summary and Design Insights

In this section, we revealed fundamental limitations of utilization-based policies, which motivated us to design a new solution to automatically adjust CPU quotas more effectively. To succeed, such a solution must satisfy the following requirements.

First, it must be **responsive to thread bursts**. Genomics applications exhibit input-dependent variability with sudden thread inflations (inflatable parallelism), requiring phase-aware signals for immediate response. Second, it must be **robust under contention**. When applications are co-located, diverse stalls (CPU vs. I/O) and millisecond-scale repetitive idle bursts make utilization noisy and misleading. A robust system must rely on stall-agnostic signals and operate at fine-grained intervals (*e.g.*, 5 ms) to avoid misallocation. Finally, it must be **adaptive to workflow dynamics**. Workflow execution paths can change at runtime due to conditional branching, activating different applications depending on input size and/or characteristics. An adaptive system continuously monitors per-application threads and adjusts quotas accordingly, eliminating the need for costly offline profiling.

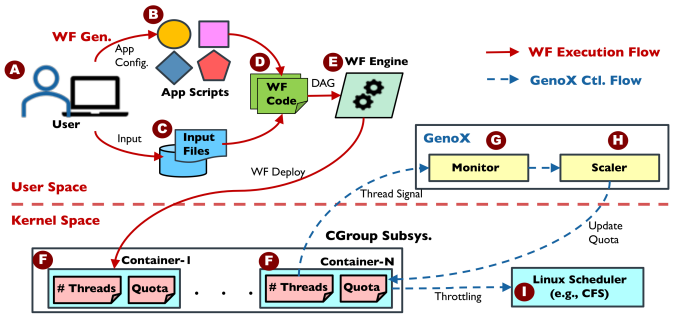


Fig. 7. **GENOX architecture.** GENOX *operates alongside the workflow engine as a sidecar: its monitor collects thread counts, the scaler computes proportional CPU quotas, and quotas are enforced via Linux cgroups and the kernel's CFS scheduler while applications run inside containers.*

IV. GENOX DESIGN

We design GENOX to address the challenges in §III. As shown in Fig. 7, GENOX operates as a sidecar alongside the workflow engine, monitoring genomics applications and dynamically adjusting CPU quotas at runtime. It includes three core components: a thread-based scaler (§IV-B), a fine-grained monitor (§IV-C), and a container-native controller (§IV-D).

A. GENOX Architecture Overview

Fig. 7 illustrates how GENOX integrates into end-to-end genomics workflow execution. A workflow begins when a user or bioinformatician (A) prepares application scripts (B) and provides genomics input files (C). These are combined into workflow code (D), which is executed by the workflow engine (E) to launch applications inside containers (F).

In parallel, GENOX operates as a sidecar component that continuously monitors the execution. Its monitor (G) collects per-container thread signals at fine-grained 5 ms intervals. The scaler (H) then computes proportional CPU quotas – approximately one core per thread – and applies them via Linux cgroup interfaces. Finally, the kernel's scheduler (I) enforces these quotas transparently, without requiring modifications to the workflow engine or applications. The next sections describe each of GENOX's components.

B. Thread-based Scaler for Responsiveness

Choosing the right scaling signal is crucial to GENOX design, as the signal determines when and how much CPU allocation should change to match compute demand. The signal must accurately reflect true demand so GENOX can allocate resources proportionally without performance loss.

Thread count as a scaling signal. GENOX uses thread count as its primary scaling signal, providing immediate response to phase changes. Unlike utilization-based policies that lag behind actual demand, thread count changes immediately when an application enters a new phase. This enables GENOX to detect *inflatable parallelism* (§III-A), where applications

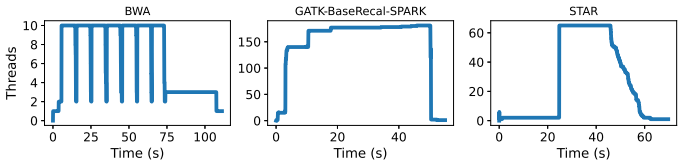


Fig. 8. **Thread-level dynamics in genomics applications.** *In BWA, STAR, and GATK, thread activity shifts sharply across phases, exposing bursts of parallelism (inflatable parallelism) that utilization signals fail to capture.*

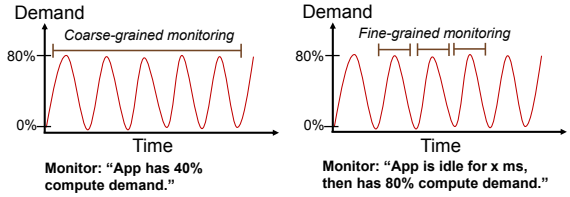


Fig. 9. **Coarse-grained vs fine-grained monitoring.** *Utilization fluctuates with millisecond-scale idle bursts, while thread counts remain stable, enabling* GENOX *to base quota decisions on reliable signals.*

spawn additional threads in proportion to user-specified settings. By tracking thread counts directly, GENOX responds to bursts promptly at fine granularity. Fig. 8 shows how thread counts clearly delineate execution phases in representative genomics applications (*e.g.*, BWA, STAR, and GATK).

Mechanism. To implement this, GENOX obtains two kernel-level metrics for each container: the number of active threads and the assigned CPU quota, both accessible through standard cgroup interfaces (*e.g.*, '/sys/fs/cgroup') with negligible overhead. Using this information, the scaler applies an empirically validated proportional heuristic that allocates approximately one CPU core per active thread. This approach balances responsiveness and control overhead while aligning with the Linux scheduler's granularity. For example, if ten threads are active, the application receives a quota for ten cores; if only two threads remain, the quota is reduced accordingly.

The monitor and scaler operate as a producer-consumer loop forming GENOX's core feedback mechanism. The monitor (producer) samples per-container thread counts at a short intervals, while the scaler (consumer) recomputes quotas accordingly. This design ensures timely adjustments with minimal overhead, as thread counts are accessed efficiently via cgroup interfaces. The same feedback pattern extends to contention handling in the fine-grained monitor (§IV-C).

Impact. By leveraging thread-level signals, GENOX quickly responds to thread bursts. Unlike utilization-based policies that provide only delayed feedback, GENOX captures phase shifts immediately and reallocates CPU quota promptly at fine granularity. This capability allows GENOX to handle inflatable parallelism effectively, ensuring that critical-path applications receive sufficient CPU resources to make forward progress without bottlenecks.

C. Fine-grained Thread Monitor for Robustness

When multiple genomics jobs execute concurrently, contention for shared resources leads to performance variability. Some phases are compute-bound while others are dominated by I/O stalls, creating heterogeneous stall patterns and millisecond-scale idle bursts (§III-B). For example, short idle bursts can make utilization appear low even though applications continue to demand CPU time. Utilization-based policies that rely on this unstable signal risk under-allocating quota during critical phases. To ensure robustness, GENOX monitors thread counts as a stable signal that is unaffected by whether stalls originate from computation or I/O.

Mechanism. This fine-grained monitor reuses the core feedback loop from §IV-B, but specializes it for high-resolution sampling under contention. In particular, it records per-container thread counts at a 5 ms interval, which is short enough to capture rapid idle bursts while remaining lightweight. Because active threads remain visible even when blocked on I/O, thread counts provide a stall-agnostic and timely reflection of demand. These measurements are continuously streamed to the scaler for quota adjustment. Fig. 9 illustrates this behavior: while coarse-grained monitoring aggregates idle bursts with compute phases making utilization appear moderate, fine-grained monitoring at 5 ms intervals captures the true demand pattern. Thread counts remain stable even as utilization oscillates, enabling GENOX to base decisions on reliable signals.

Impact. By grounding quota decisions in stable thread-level signals, GENOX achieves robustness under contention. It avoids misinterpreting idle bursts as reduced demand and sustains fair allocation across co-located genomics applications, thereby mitigating contention-induced slowdowns.

D. Container-native Controller for Adaptiveness

Genomics workflows frequently undergo dynamic branching, where execution paths depend on input data or intermediate results (§III-C). As a result, new containers may be created on-the-fly, while others may terminate earlier than expected. A static quota plan cannot anticipate such changes, leading to wasted resources or starvation of newly tasks. To remain adaptive, GENOX must handle workflow dynamics without manual intervention or workflow engine modifications.

Mechanism. GENOX achieves this adaptiveness through a container-native controller that integrates directly with Linux cgroups. When a workflow engine launches new tasks, they automatically appear as containers within the cgroup hierarchy. The controller detects these containers in real time and applies GENOX's quota policies without requiring any workflow-level awareness. Similarly, when containers terminate, their quotas are immediately return to the pool.

The controller operates as a lightweight daemon that monitors the cgroup subsystem for container events. Upon detecting a new container, it attaches quota enforcement hooks so that scaling decisions from the monitor and scaler (§IV-B, §IV-C) are applied transparently. Because it leverages existing OS primitives, the controller incurs negligible overhead and requires no modifications to workflow engines or applications.

Impact. By coupling quota enforcement to containers, GENOX achieves adaptiveness to workflow dynamics. It automatically incorporates new tasks, releases resources from

completed ones, and sustains efficient utilization throughout branching workflows. This design enables GENOX to provide adaptive CPU allocation without requiring changes to genomics applications or workflow engines.

V. EVALUATION RESULTS OF GENOX

We evaluate GENOX in production-like settings using production-scale datasets and complete DNA-Seq and RNA-Seq pipelines. Our evaluation assesses three criteria: (i) **responsiveness** to bursty, phase-shifting demand, (ii) **robustness** under contention and I/O stalls, and (iii) **adaptiveness** to workflow dynamics.

GENOX is compared against three representative baselines, capturing widely used heuristics for dynamic CPU allocation. Evaluations cover both micro-benchmarks (case studies) and macro-scale results. The case studies (§V-B) validate GENOX's mechanisms on representative applications, while macro-scale experiments (§V-C, §V-D) quantify application-level and workflow-level improvements under realistic contention. We report execution time, normalized speedup, straggler slowdown, and end-to-end workflow runtime.

A. Evaluation Methodology

HW setup. All experiments are conducted on the Chameleon NSF Cloud testbed, using a server equipped with 96 CPU cores, 200 GB of memory, and 500 GB of SSD storage. Workflows are containerized and orchestrated via Nextflow.

Baselines. We evaluate GENOX against three baselines widely studied in dynamic CPU allocation.

Autopilot [13] monitors recent CPU utilization based on its moving-window averaging strategy and adjusts quotas based on smoothed historical demand. Autopilot's approach reduces short-term fluctuations and avoids overreacting to momentary noise, but it also delays responses to sudden bursts or phase shifts.

AutoThrottle [14] employs reactive scaling to adjust CPU quotas aggressively once utilization crosses predefined thresholds. AutoThrottle responds quickly to bursts but is prone to overshoot and instability when demand fluctuates.

SHOWAR [15] uses statistical thresholding (*e.g.*, three-sigma rules) to identify deviations from average utilization. SHOWAR aims to be robust against noise but often underestimates demand in bursty workloads, leading to slowdowns due to stragglers.

Evaluation metrics. We use metrics at both application and workflow levels. For application-level studies, we measure per-application execution time and and straggler slowdown. To capture straggler effects accurately, we evaluate the completion time of the last-finishing workflow, reflecting the cumulative impact of contention across constituent applications. For workflow-level studies, we measure end-to-end runtime and speedup of complete DNA-Seq and RNA-Seq pipelines, showing how per-stage allocation impacts overall throughput in production workflows.

Table 2. **CPU usage patterns of three representative genomics applications.** *Each application is selected to demonstrate a particular design choice of* GENOX. *i.e., responsiveness (BWA), responsiveness and robustness (STAR), and adaptivity (BamSorMaDup).*

App	Exec. Pattern (Thread/CPU)	Eval. Focus
BWA	Bursty inflatable parallelism with sudden thread inflation	Responsive.
STAR	Multi-phase execution with rising /falling threads and idle gaps	Responsive. + Robust.
BamSor-MaDup	Constant parallel execution with fixed 16-thread demand	Adaptive.

B. Micro-benchmarks (Case Studies)

To validate GENOX's mechanisms under controlled execution patterns, we conduct microbenchmarks on three representative genomics applications: BWA, STAR, and BamSorMaDup. Table 2 summarizes how each application corresponds to the challenges (in §III) and evaluates a specific GENOX's capability: responsiveness, robustness, and adaptiveness.

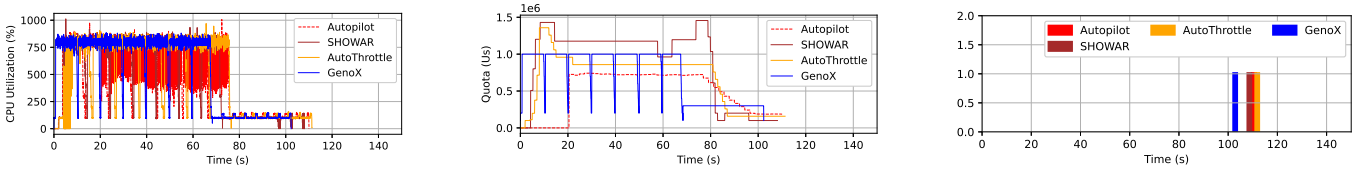
BWA (Responsiveness). We use BWA, a DNA aligner [26], to stress GENOX's ability to handle *bursty inflatable parallelism* (§III-A). The key question is whether GENOX can adjust quotas fast enough to prevent throttling during bursts.

Fig. 10a shows CPU utilization under different policies. Utilization-based approaches (Autopilot, SHOWAR, AutoThrottle) lag behind bursts, reacting only after resource consumption changes. GENOX prevents throttling by tracking threads directly. Fig. 10b reports CPU quota allocation over time. GENOX increases CPU quota promptly in proportion to thread inflation and scales down once bursts decline. In contrast, utilization-based baselines exhibit delayed reactions. For example, Autopilot, SHOWAR, and AutoThrottle increase quotas only after demand has shifted, resulting in temporary under-allocation. As a result, GENOX minimizes throttling penalties and achieves the fastest completion time among all policies (Fig. 10c).

STAR (Responsiveness + Robustness). We use STAR, an RNA-seq aligner [6] to test GENOX under *multi-phase execution with idle bursts and heterogeneous stalls* (§III-B). The key challenge is whether GENOX can remain responsive to bursts while being robust against misleading I/O stall signals.

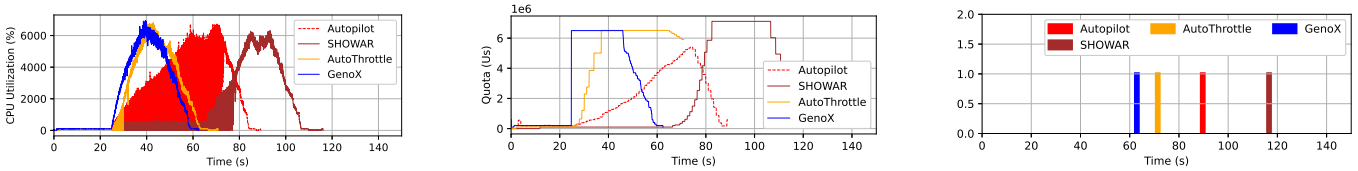
Fig. 11a shows CPU utilization across policies. Baseline approaches overestimate demand during idle bursts and underestimate it when demand rises quickly. However, GENOX, relying on thread counts rather than utilization, avoids these misinterpretations and maintains a closer alignment with actual demand. Fig. 11b reports quota allocation. GENOX scales quotas up immediately as thread counts rise and reduces them once phases complete. In contrast, baselines lag behind demand by adjusting CPU quota only after consumption has shifted. As a result, GENOX achieves the fastest completion time (Fig. 11c). By preventing over-throttling during bursts and avoiding over-provisioning during idle gaps, GENOX reduces wasted resources and improves efficiency relative to all baselines.

BamSorMaDup (Adaptiveness). We use BamSorMaDup, a duplicate-marking tool [27], to evaluate GENOX's *adaptiveness under steady parallel workloads*, where the thread count



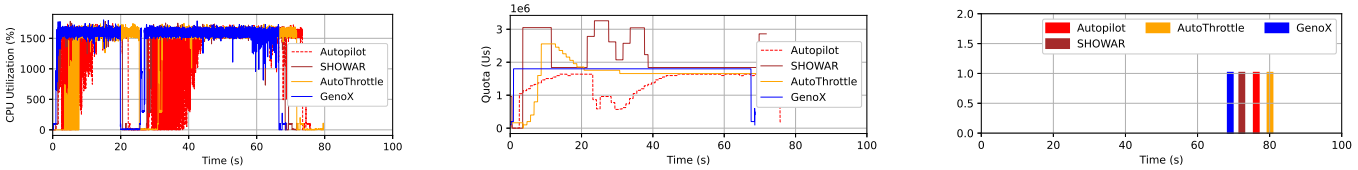
(a) CPU utilization (BWA) (b) CPU quota allocation (BWA) (c) BWA finish times across policies

Fig. 10. **BWA case study results.** *'(a) CPU utilization:' Baselines lag behind bursts while* GENOX *immediately detects thread inflation. '(b) Quota allocation over time:'* GENOX *scales up/down promptly with thread count whereas baselines react after demand shifts. '(c) Completion times:'* GENOX *achieves the shortest runtime by eliminating/minimizing throttling penalties, while baselines finish later due to delayed reactions.*



(a) CPU utilization (STAR) (b) CPU quota allocation (STAR) (c) STAR finish times across policies

Fig. 11. **STAR case study results.** *(a) 'CPU utilization:' Baselines misinterpret idle bursts as reduced demand while* GENOX *tracks thread counts to maintain accurate allocation. (b) 'Quota allocation over time:'* GENOX *responds immediately to thread changes, whereas baselines exhibit delayed reactions. (c) 'Completion times:'* GENOX *achieves fastest runtime through prompt response and robustness to idle bursts.*



(a) CPU utilization (BamSorMaDup) (b) CPU quota allocation (BamSorMaDup) (c) BamSorMaDup finish times across policies

Fig. 12. **BamSorMaDup case study results.** *'(a) CPU utilization:' UBaselines fluctuate due to I/O stalls while* GENOX *maintains stability matching the constant 16-thread profile. '(b) Quota allocation over time:'* GENOX *consistently assigns one core per thread whereas baselines make unstable adjustments. '(c) Completion times:' All policies finish similarly but* GENOX *avoids resource waste, demonstrating efficient adaptation to steady workloads.*

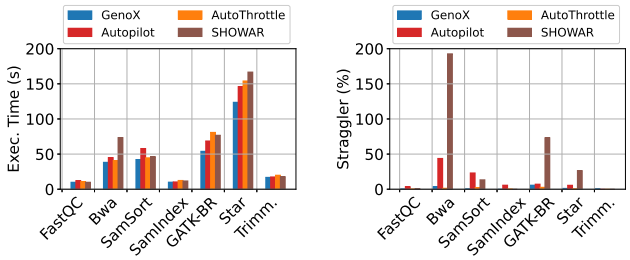
remains constant at 16 throughout execution (§III-C). The challenge here is whether GENOX can avoid wasting resources in the absence of bursts or dynamic branching.

Fig. 12a shows CPU utilization. Baseline policies occasionally misinterpret I/O stalls, leading to oscillations in quota allocation. GENOX, by contrast, detects the constant 16-thread demand and keeps allocations minimal and stable. Fig. 12b reports CPU quota allocation. GENOX consistently assigns one core per thread (16 cores total), matching the constant-thread profile. However, the baselines make unstable quota adjustments due to noisy utilization signals, leading to inefficient resource allocation. As shown in Fig. 12c, all policies finish within a similar time frame since BamSorMaDup is not CPU-intensive. However, GENOX achieves this without wasting CPU cycles, demonstrating its efficiency and adaptiveness.

Takeaway. The case studies show that GENOX addresses three primary challenges: immediate response to thread bursts (case BWA); maintaining stability in the presence of I/O-induced noise (case STAR); and efficient adaptation to constant-thread workloads (case BamSorMaDup).

C. Application-level Performance

We next evaluate GENOX at the application level to assess its performance under resource contention. To introduce contention, we execute multiple concurrent workflows, where each



(a) Execution Times (b) Straggler Slowdown

Fig. 13. **Execution times (13a) and straggler slowdown (13b) of DNA-Seq under contention.** *'(a) Execution time:'* GENOX *consistently reduces completion times across all applications, whereas baselines show inconsistent performance. '(b) Straggler slowdown:'* GENOX *prevents performance degradation from stragglers, whereas baselines suffer severe delays (e.g., 200% slowdown for BWA under SHOWAR and 43% under Autopilot.)*

workflow runs several genomics applications that compete for shared computing resources. This experimental setup reflects production environments, where co-running tasks are prevalent and straggler effects can significantly impact overall workflow performance. We measure performance using the completion time of the last-finishing workflow, which captures the impact of contention across all constituent applications.

Fig. 13a shows the execution times of DNA-Seq applications under contention. GENOX consistently achieves the shortest completion times and outperforms all baseline meth-

Table 3. **End-to-end execution times (sec.) for DNA-Seq and RNA-Seq workflows.** GENOX *consistently achieves the shortest runtimes, speeding up DNA-Seq by 12 – 18% and RNA-Seq by 19 – 25% compared to baselines.*

	Autopilot	AutoThrottle	SHOWAR	GENOX
DNA-Seq	1085	1115	1059	944
RNA-Seq	339	352	335	282

ods by effectively adapting to varying CPU usage patterns across applications. In contrast, baseline approaches show inconsistent performance across applications. For example, while SHOWAR achieves the second-best performance for Trimmomatic, it performs worst for BWA. Similarly, Autopilot ranks second-best for Samtools-Index but exhibits the poorest performance for Samtools-Sort. Across all evaluated applications, GENOX achieves the fastest completion times, reducing execution time by up to 49% relative to baseline methods, with an average improvement of 15.7%.

We also measured the impact of stragglers. Fig. 13b presents straggler slowdowns, defined as the additional wait time imposed on applications in the slowest workflow relative to those in the fastest (*e.g.*, if the fastest finishes at $t = 8$ and the slowest at $t = 10$, the slowdown is $(10–8)/8 = 25\%$). GENOX effectively avoids such slowdowns, while utilization-based baselines suffer from significant delays. SHOWAR, for example, struggles with BWA, GATK-BR, and STAR, exhibiting nearly 200% slowdown on BWA, where the faster workflow waits almost its entire runtime. Autopilot performs better but still incurs notable slowdowns, including 43% on BWA and 23% on Samtools-Sort. In contrast, GENOX prevents these effects by adjusting quotas promptly, avoiding the accumulation of straggler delays.

Takeaway. At the application level, GENOX consistently outperforms baselines and effectively handles contention by: (1) maintaining consistent performance across diverse applications, (2) minimizing straggler effects through prompt quota adjustments, and (3) achieving significant end-to-end workflow speedups, validating GENOX's thread-based design.

D. Workflow-level Performance

We execute complete DNA-Seq and RNA-Seq pipelines with production-scale datasets to measure GENOX's workflow-level performance. This evaluation is to capture the cumulative effects of CPU quota allocation decisions across multiple workflow stages. In genomics pipelines, delays in a single application can propagate through dependent tasks, leading to significant increases in overall workflow runtime.

Table 3 reports total workflow execution times. GENOX consistently achieves the shortest runtimes by reducing completion time by 12% – 18% for DNA-Seq and 19% – 25% for RNA-Seq compared to baselines. At the workflow level, GENOX was able to maintain the improvements observed at the application level, with average speedups comparable to per-application benefits. Moreover, GENOX was the only approach that kept performance close to unconstrained execution while avoiding the inefficiencies of wasted resources.

Fig. 14 and 15 report the start and finish times for individual applications within both workflows. Baselines often prolong

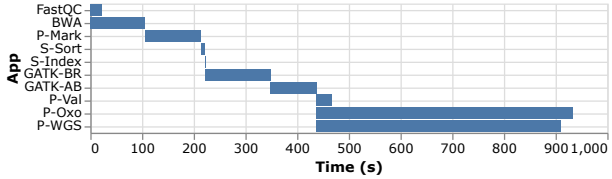


Fig. 14. **Per-application start and finish times in DNA-Seq.** *Baseline policies prolong key stages (e.g., P-OxO (Picard-CollectOxoGMetrics) by up to 26%), delaying downstream tasks and inflating overall runtime, while* GENOX *keeps stage completion times shorter and more balanced.*

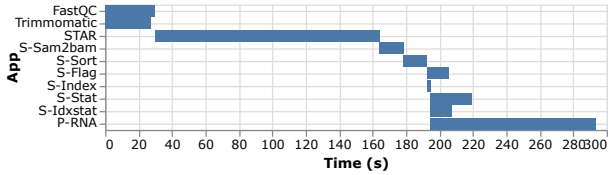


Fig. 15. **Per-application start and finish times in RNA-Seq.** *Baselines extend long-running stages such as STAR (by 19% under SHOWAR), which cascades into delays for dependent tasks.* GENOX *reduces such stage-level bottlenecks, sustaining faster progression at the workflow level.*

long-running stages, delaying dependent tasks and inflating overall runtime. For example, STAR in RNA-Seq extends by 19% under SHOWAR, while P-OxO (PicardCollectQxqGMetrics) [48] in DNA-Seq is prolonged by 26%. These stage-level delays compound throughout the workflow, explaining the performance gaps in Table 3.

Takeaway. At the workflow level, GENOX manages prompt quota adjustments throughout the entire pipeline. By doing so, GENOX avoids the cascading delays that occur when one stage's slowdown impacts subsequent stages, delivering practical benefits for large-scale genomics processing.

VI. DISCUSSION

We discuss GENOX's effective scenarios and limitations.

Effective Scenarios for GENOX. Our evaluation shows that GENOX achieves the highest effectiveness in dynamic workflows with significant resource contention, which are frequently encountered in genomics pipeline workloads. When applications exhibit bursty parallelism or undergo phase shifts, GENOX leverages thread-level signals to respond immediately, thereby eliminating the lag that is often observed in utilization-based policies. Its fine-grained monitor prevents idle bursts from being incorrectly interpreted as reduced resource demand, which ensures stable and accurate CPU quota allocation throughout workflow execution.

These capabilities are especially advantageous in multi-threaded applications (*e.g.*, BWA, STAR, GATK), which frequently alternate between CPU- and I/O-bound phases. Under these conditions, GENOX sustains high throughput without manual tuning, consistently outperforming utilization-based approaches by reducing application completion times and shortening the overall execution time of end-to-end workflows.

Limitations of GENOX. While GENOX is effective in many settings, its benefits *may* diminish under certain conditions.

First, GENOX's reliance on thread count as its primary scaling signal offers limited advantages for single-threaded

Table 4. **Execution times of DNA-Seq and RNA-Seq workflows without contention.** *In resource-rich environments,* GENOX *performs nearly identically to Linux's NoLimit policy, with only about 1% difference.*

	NoLimit	GENOX	Differences
DNA-Seq	935	944	0.5% ↑; GENOX is slower
RNA-Seq	286	282	1.4% ↓; GENOX is faster

applications. When execution is inherently serial, GENOX performs similarly to Linux defaults by allocating only one core. To address this limitation, we plan to incorporate additional scaling signals. The Linux kernel exposes useful scheduler statistics via '/sys/fs/' and '/proc/' interfaces, which can improve demand estimation. However, some of these metrics require kernel-level access. This introduces a trade-off between preserving GENOX's non-invasive design and enabling finer-grained scaling decisions.

Second, GENOX offers minimal benefits in non-contented environments. To quantify this, we compared GENOX against Linux's greedy (NoLimit) policy, which allows unrestricted core usage. When executing DNA-Seq and RNA-Seq without contention, GENOX was only about 1% slower than NoLimit (able 4). This confirms that while GENOX's thread monitoring and quota enforcement incur negligible overhead, they provide no advantage when contention is absent. This limitation is expected, as GENOX is designed for co-located production environments where workflows compete for shared resources.

VII. RELATED WORK

CPU quota allocation for parallel applications has been extensively studied in HPC and cloud domains. We categorize prior work based on their allocation strategies and compare them with GENOX in the context of genomics workloads. Table 5 summarizes this comparison.

Utilization-based Reactive Approaches. Autopilot, AutoThrottle, and SHOWAR share our cgroup-based, non-invasive design but differ in scaling signals. Being reactive, they fail to handle the inflatable parallelism (§III-A) and millisecond-scale idle bursts (§III-B), commonly observed in genomics workloads, as they adjust quotas only after utilization changes occur. We confirmed their limitations in §V.

Predictive Approaches. Nodens [16], SmartHarvest [17], and Libra [18] use prediction models to estimate future demand. While proactive, these methods require substantial training data and historical patterns. For genomics workflows characterized by input-dependent variability (§III-A) and dynamic branching (§III-C), prediction accuracy also becomes unreliable, often leading to misallocation.

Application-Integrated Approaches. Systems like Arachne [49], Shenango [50], and Caladan [51] aim to achieve microsecond-scale responsiveness through application-level integration. Arachne rewrites applications with a core-aware threading API, while Shenango and Caladan use customized runtime libraries to reduce latency while maintaining POSIX compatibility. Caladan also adapts network queue depths as demand signals. These systems target latency-critical microservices with microsecond requirements and require code modifications or runtime linking, both impractical for workflows relying on numerous third-party tools.

Table 5. **Comparison of GENOX with prior CPU allocation systems for genomics workloads.** *Each system satisfies a different subset of design goals relevant to CPU quota allocation. While prior approaches target general-purpose settings (e.g., microservices), only* GENOX *jointly satisfies the responsiveness, robustness, and adaptiveness required by genomics workloads. This comparison highlights design trade-offs and genomics workload-specific suitability rather than general performance advantage.*

System	Responsiveness	Robustness	Adaptiveness
Autopilot [13]	·	·	✓
Autothrottle [14]	·	·	✓
Arachne [49]	✓	✓	·
Caladan [51]	·	✓	·
HarvestVM [19]	✓	·	·
Libra [18]	✓	·	✓
Nodens [16]	·	·	✓
PerfIso [20]	✓	·	·
Shenango [50]	✓	✓	·
SHOWAR [15]	·	·	✓
SmartHarvest [17]	✓	·	·
GENOX	✓	✓	✓

Kernel/Hypervisor-based Approaches. HarvestVM [19] and PerfIso [20] modify system software to control CPU allocation. HarvestVM operates at the hypervisor layer, reclaiming idle resources for best-effort workloads, while PerfIso pins cores at the kernel level to isolate performance. However, their control granularity, *e.g.*, tens of milliseconds, is too coarse for genomics applications. Both approaches also require privileged access, which is typically unavailable in shared clusters.

Key differences. GENOX differs from these systems in three fundamental ways. First, it uses thread counts rather than utilization or predictions as a stall-agnostic signal, enabling immediate CPU quota adjustments. Second, it performs fine-grained monitoring at 5 ms intervals, capturing millisecond-scale dynamics without microsecond-level overhead. Finally, it requires no modifications to applications, runtimes, or kernels, relying solely on standard Linux cgroup interfaces for deployment. Table 5 shows that sys uniquely satisfies all three design goals – *responsiveness*, *robustness*, and *adaptiveness* – within a single production-ready system, making it well suited for containerized genomics workflows.

VIII. CONCLUSION

GENOX is a dynamic CPU quota allocator specifically designed for genomics workflows in shared clusters. We first revealed three key requirements for effective quota management: responsiveness to thread bursts, robustness under contention, and adaptiveness to workflow dynamics. GENOX meets these requirements by leveraging thread counts as stall-agnostic scaling signals and coordinating three lightweight components: (i) a thread-based scaler, (ii) a fine-grained monitor operating at 5 ms intervals, and (iii) a container-native controller built on standard cgroups. Experiments on DNA-Seq and RNA-Seq workflows show that GENOX significantly reduces execution time by up to 50%, mitigates straggler slowdowns, and improves end-to-end workflow runtime by over 20% compared to state-of-the-art baselines, demonstrating its effectiveness for production-scale genomics workloads.

References

[1] Martin L. Putra, In Kee Kim, Haryadi S. Gunawi, and Robert L. Grossman. CNT: Semi-Automatic Translation from CWL to Nextflow for Genomic Workflows. In *The 23rd IEEE International Conference on Bioinformatics and Bioengineering (BIBE)*, Virtual, November, 2023.

[2] Fabian Lehmann, Jonathan Bader, Friedrich Tschirpke, Lauritz Thamsen, and Ulf Leser. How Workflow Engines Should Talk to Resource Managers: A Proposal for a Common Workflow Scheduling Interface. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, May, 2023.

[3] Md. Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. Efficient Architecture-Aware Acceleration of BWA-MEM for Multicore Systems. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May, 2019.

[4] Michael Hao Tong, Robert L. Grossman, and Haryadi S. Gunawi. Experiences in Managing the Performance and Reliability of a Large-Scale Genomics Cloud Platform. In *USENIX Annual Technical Conference (ATC)*, July, 2021.

[5] Sergei Yakneen, Sebastian M. Waszak, Michael Gertz, and Jan O. Korbel. Butler Enables Rapid Cloud-Based Analysis of Thousands of Human Genomes. *Nature Biotechnology*, 38:288 – 292, 2020.

[6] Alexander Dobin, Carrie A. Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R. Gingeras. STAR: Ultrafast Universal RNA-Seq Aligner. *Bioinformatics*, 29(1):15–21, 10 2012.

[7] Genome Analysis Toolkit. https://gatk.broadinstitute.org/hc/en-us.

[8] Azza E. Ahmed, Joshua M. Allen, Tajesvi Bhat, Prakruthi Burra, Christina E. Fliege, Steven N. Hart, Jacob R. Heldenbrand, Matthew E. Hudson, Dave Deandre Istanto, Michael T. Kalmbach, Gregory D. Kapraun, Katherine I. Kendig, Matthew Charles Kendzior, Eric W. Klee, Nate Mattson, Christian A. Ross, Sami M. Sharif, Ramshankar Venkatakrishnan, Faisal M. Fadlelmola, and Liudmila S. Mainzer. Design Considerations for Workflow Management Systems Use in Production Genomics Research and the Clinic. *Scientific Reports*, 11(1):21680, 2021.

[9] T. M. A. Do, Loïc Pottier, R. Ferreira da Silva, F. Suter, S. Caíno-Lores, M. Taufer, E. Deelman, and T. Peterka. Accelerating Scientific Workflows on HPC Platforms with In Situ Processing. In *IEEE/ACM 22nd International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, May, 2022.

[10] Taylor Reiter, Phillip T Brooks, Luiz Irber, Shannon E K Joslin, Charles M Reid, Camille Scott, C Titus Brown, and N Tessa Pierce-Ward. Streamlining Data-Intensive Biology with Workflow Systems. *GigaScience*, 10(1):giaa140, 2021.

[11] FastQC. https://www.bioinformatics.babraham.ac.uk/projects/fastqc/.

[12] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. Trimmomatic: a Flexible Trimmer for Illumina Sequence Data. *Bioinformatics*, 30(15):2114–2120, 2014.

[13] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload Autoscaling at Google. In *Fifteenth EuroSys Conference (EuroSys)*, April, 2020.

[14] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y. Yan. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, April, 2024.

[15] Ataollah Fatahi Baarzi and George Kesidis. SHOWAR: Right-Sizing And Efficient Scheduling of Microservices. In *ACM Symposium on Cloud Computing (SoCC)*, November, 2021.

[16] Jiuchen Shi, Hang Zhang, Zhixin Tong, Quan Chen, Kaihua Fu, and Minyi Guo. Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters. In *USENIX Annual Technical Conference (2023)*, July, 2023.

[17] Yawen Wang, Kapil Arya, Marios Kogias, Manohar Vanga, Aditya Bhandari, Neeraja J. Yadwadkar, Siddhartha Sen, Sameh Elnikety, Christos Kozyrakis, and Ricardo Bianchini. SmartHarvest: Harvesting Idle CPUs Safely and Efficiently in the Cloud. In *Sixteenth European Conference on Computer Systems (EuroSys)*, April, 2021.

[18] Hanfei Yu, Christian Fontenot, Hao Wang, Jian Li, Xu Yuan, and Seung-Jong Park. Libra: Harvesting Idle Resources Safely and Timely in Serverless Clusters. In *The 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, June, 2023.

[19] Pradeep Ambati, Iñigo Goiri, Felipe Vieira Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, Marcus Fontoura, and Ricardo Bianchini. Providing SLOs for Resource-Harvesting VMs in Cloud Platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November, 2020.

[20] Calin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek R. Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, and Junhua Wang. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *USENIX Annual Technical Conference (ATC)*, July, 2018.

[21] Jonathan Bader, Joel Witzke, Soeren Becker, Ansgar Lößer, Fabian Lehmann, Leon Doehler, Anh Duc Vu, and Odej Kao. Towards Advanced Monitoring for Scientific Workflows. In *IEEE International Conference on Big Data (Big Data)*, December, 2022.

[22] Jonathan Bader, Lauritz Thamsen, Svetlana Kulagina, Jonathan Will, Henning Meyerhenke, and Odej Kao. Tarema: Adaptive Resource Allocation for Scalable Scientific Workflows in Heterogeneous Clusters. In *IEEE International Conference on Big Data (Big Data)*, December, 2021.

[23] Alok Singh, Shweta Purawat, Arvind Rao, and Ilkay Altintas. Modular Performance Prediction for Scientific Workflows Using Machine Learning. *Future Generation Computing Systems*, 114:1–14, 2021.

[24] Ewa Deelman, Anirban Mandal, Ming Jiang, and Rizos Sakellariou. The Role of Machine Learning in Scientific Workflows. *International Journal of High Performance Computing Applications*, 33(6), 2019.

[25] Overview of GDC Harmonization Workflows. https://github.com/NCI-GDC/gdc-workflow-overview.

[26] Heng Li and Richard Durbin. Fast and Accurate Short Read Alignment with Burrows–Wheeler Transform. *Bioinformatics*, 25(14):1754–1760, 05 2009.

[27] German Tischler and Steven Leonard. Biobambam: Tools for Read Pair Collation-Based Algorithms on BAM Files. *Source Code for Biology and Medicine*, 9:13, 2014.

[28] Nextflow. https://www.nextflow.io/docs/latest/.

[29] Common Workflow Language User Guide. https://www.commonwl.org/user_guide/.

[30] Oleksandr Rudyy, Marta Garcia-Gasulla, Filippo Mantovani, Alfonso Santiago, Raül Sirvent, and Mariano Vázquez. Containers in HPC: A Scalability and Portability Study in Production Biological Simulations. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May, 2019.

[31] Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic Local Alignment Search Tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

[32] Stepanka Zverinova and Victor Guryev. Variant Calling: Considerations, Practices, and Developments. *Human Mutation*, 43(8):976–985, 2022.

[33] David C. Samuels, Hui Yu, and Yan Guo. Is It Time to Reassess Variant Annotation? *Trends in Genetics*, 38(6):521–523, 2022.

[34] OpenWDL. https://openwdl.org/.

[35] Cromwell. https://cromwell.readthedocs.io/en/latest/.

[36] cwltool: The reference implementation of the common workflow language standards. https://github.com/common-workflow-language/cwltool.

[37] Moiz Arif, Avinash Maurya, M. Mustafa Rafique, Dimitrios S. Nikolopoulos, and Ali Raza Butt. Application-Attuned Memory Management for Containerized HPC Workflows. In *IEEE International Parallel and Distributed Processing Symposium*, May, 2024.

[38] Sergey Nurk, Sergey Koren, Arang Rhie, Mikko Rautiainen, Andrey V. Bzikadze, Alla Mikheenko, Mitchell R. Vollger, Nicolas Altemose, Lev Uralsky, Ariel Gershman, Sergey Aganezov, Savannah J. Hoyt, Mark Diekhans, Glennis A. Logsdon, Michael Alonge, Stylianos E. Antonarakis, Matthew Borchers, Gerard G. Bouffard, Shelise Y. Brooks, Gina V. Caldas, Nae-Chyun Chen, Haoyu Cheng, Chen-Shan Chin, William Chow, Leonardo G. de Lima, Philip C. Dishuck, Richard Durbin, Tatiana Dvorkina, Ian T. Fiddes, Giulio Formenti, Robert S. Fulton, Arkarachai Fungtammasan, Erik Garrison, Patrick G. S. Grady, Tina A. Graves-Lindsay, Ira M. Hall, Nancy F. Hansen, Gabrielle A. Hartley, Marina Haukness, Kerstin Howe, Michael W. Hunkapiller, Chirag Jain, Miten Jain, Erich D. Jarvis, Peter Kerpedjiev, Melanie Kirsche, Mikhail Kolmogorov, Jonas Korlach, Milinn Kremitzki, Heng Li, Valerie V. Maduro, Tobias Marschall, Ann M. McCartney, Jennifer McDaniel, Danny E. Miller, James C. Mullikin, Eugene W. Myers, Nathan D. Olson, Benedict Paten, Paul Peluso, Pavel A. Pevzner, David

Porubsky, Tamara Potapova, Evgeny I. Rogaev, Jeffrey A. Rosenfeld, Steven L. Salzberg, Valerie A. Schneider, Fritz J. Sedlazeck, Kishwar Shafin, Colin J. Shew, Alaina Shumate, Ying Sims, Arian F. A. Smit, Daniela C. Soto, Ivan Sović, Jessica M. Storer, Aaron Streets, Beth A. Sullivan, Françoise Thibaud-Nissen, James Torrance, Justin Wagner, Brian P. Walenz, Aaron Wenger, Jonathan M. D. Wood, Chunlin Xiao, Stephanie M. Yan, Alice C. Young, Samantha Zarate, Urvashi Surti, Rajiv C. McCoy, Megan Y. Dennis, Ivan A. Alexandrov, Jennifer L. Gerton, Rachel J. O'Neill, Winston Timp, Justin M. Zook, Michael C. Schatz, Evan E. Eichler, Karen H. Miga, and Adam M. Phillippy. The Complete Sequence of a Human Genome. *Science*, 376(6588):44–53, 2022.

[39] Dong H. Ahn, Xiaohua Zhang, Jeffrey Mast, Stephen Herbein, Francesco Di Natale, Dan Kirshner, Sam Ade Jacobs, Ian Karlin, Daniel J. Milroy, Bronis R. de Supinski, Brian Van Essen, Jonathan E. Allen, and Felice C. Lightstone. Scalable Composition and Analysis Techniques for Massive Scientific Workflows. In *IEEE International Conference on e-Science (e-Science)*, October, 2022.

[40] Xiangqun Zheng-Bradley, Ian Streeter, Susan Fairley, David Richardson, Laura Clarke, Paul Flicek, and the 1000 Genomes Project Consortium. Alignment of 1000 Genomes Project Reads to Reference Assembly GRCh38. *GigaScience*, 6(7):gix038, 05 2017.

[41] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux Utility for Resource Management. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, June, 2003.

[42] Robert L. Henderson. Job Scheduling Under the Portable Batch System. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, April, 1995.

[43] Garrick Staples. TORQUE - TORQUE resource manager. In *ACM/IEEE Conference on High Performance Networking and Computing (SC)*, November, 2006.

[44] Kubernetes Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.

[45] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. Apache Hadoop YARN: Yet Another Resource Negotiator. In *ACM Symposium on Cloud Computing (SoCC)*, November, 2013.

[46] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons Learned from the Chameleon Testbed. In *USENIX Annual Technical Conference (ATC)*, July, 2020.

[47] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. TMO: Transparent Memory Offloading in Datacenters. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, February, 2022.

[48] GATK - CollectOxoGMetrics (Picard). https://gatk.broadinstitute.org/hc/en-us/articles/360037227552-CollectOxoGMetrics-Picard.

[49] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John K. Ousterhout. Arachne: Core-Aware Thread Management. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October, 2018.

[50] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February, 2019.

[51] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating Interference at Microsecond Timescales. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November, 2020.